

Dəyişənlər və yaddaş

RAM, Stack, Heap, Value Type, Reference Type, new və Garbage Collector - tələbə üçün geniş izah

Bu dərstdə nəyi başa düşəcəksən: Kod yazanda dəyişənin sadəcə ad olmadığını, proqram işləyərkən məlumatın RAM-da yaşadığını, int ilə array-in niyə fərqli davrandığını və gələcək data structure mövzularında stack/heap düşüncəsinin niyə vacib olduğunu öyrənəcəksən.

Bölmə	Öyrənəcəyin əsas fikir
Dəyişən	Ad, tip və dəyər birlikdə proqramın cari vəziyyətini yaradır.
RAM	Proqram işləyəndə dəyərlər və obyektlər müvəqqəti yaddaşda saxlanılır.
Value type	Kopyalananda dəyər ayrılır.
Reference type	Kopyalananda obyekt yox, obyektə istinad paylaşılır.
Stack/Heap	Lokal dəyərlər və obyektlər haqqında sadə mental model qurmağa kömək edir.
GC	Artıq əlçatan olmayan heap obyektlərini runtime sonradan təmizləyə bilər.

1. Dəyişən nədir?

Dəyişən proqramda məlumat saxlamaq üçün istifadə etdiyimiz addır. Amma bu cümlə təkbaşına azdır. Daha düzgün düşüncə belədir: proqram işləyərkən müəyyən məlumat RAM-da saxlanılır və biz həmin məlumata kodda bir ad veririk. O ad dəyişəndir.

Məsələn:

```
int age = 20;  
string name = "Aysel";  
bool isActive = true;
```

Bu üç sətirdə hər dəyişənin üç hissəsi var: ad, tip və dəyər. age dəyişənin adıdır, int onun tipidir, 20 isə dəyəridir. Tip kompüterə bu məlumatla necə işləyəcəyini bildirir. int üzərində toplama aparmaq olar, bool şərtə istifadə edilir, string isə mətn obyektidir.

Hissə	Nümunə	Mənası
Ad	age	Kodda məlumatı çağırmaq üçün istifadə etdiyimiz simvol.
Tip	int	Məlumatın formasını və onunla hansı əməliyyatların mümkün olduğunu bildirir.
Dəyər	20	Proqram işləyərkən saxlanılan konkret məlumat.

Yadda saxla: Dəyişən sadəcə kodda gördüyün söz deyil. O, proqram işləyərkən saxlanılan vəziyyətə çıxış nöqtəsidir.

Özünü yoxla

- price dəyişəni decimal tipindədirsə, adı, tipi və dəyəri necə ayırırsan?
- Niyə hər məlumat üçün eyni tipdən istifadə etmirik?
- Dəyişənin dəyəri proqram işləyəndə dəyişə bilərmi?

2. RAM nədir və niyə vacibdir?

RAM proqramın işləmə vaxtı istifadə etdiyi sürətli və müvəqqəti yaddaşdır. Sənin yazdığın .cs faylı diskdə saxlanılır. Amma proqram işə düşəndə onun cari dəyərləri, obyektləri, array-ləri və metod çağırışları RAM-la əlaqəli olur.

Bunu belə düşün: disk kitab rəfidir, RAM isə dərs zamanı masanın üstünə açdığın dəftərdir. Kitab rəfdə qalır, amma sən işləyərkən aktiv məlumat masanın üstündə olur. Proqram bağlananda həmin aktiv vəziyyət itir.

Anlayış	Sadə izah	Nümunə
Disk	Faylların qalıcı saxlandığı yer.	.cs faylı, şəkil, database faylı.
RAM	Proqram işləyərkən istifadə olunan müvəqqəti yaddaş.	Dəyişənlər, obyektlər, array-lər.
CPU	Əməliyyatları icra edən hissə.	Toplama, müqayisə, şərt yoxlama.
Backend üçün niyə önəmlidir: Server proqramında minlərlə request eyni anda obyekt yarada bilər. Əgər sən RAM və reference anlayışını başa düşmürsənsə, böyük list-lərin, cache-in, lazımsız obyekt saxlamanın və memory leak problemlərinin səbəbini anlamaq çətin olacaq.		

Məsələn, API istifadəçi məlumatını oxuyanda User obyektini yarada bilər. Çoxlu istifadəçi, çoxlu obyekt deməkdir. Bu obyektlərin nə vaxt yaranması, nə qədər yaşaması və kim tərəfindən göstərilməsi backend performansına təsir edir.

3. Value Type: dəyərin özü ilə işləmək

Value type dəyişəndə dəyərin özü ilə işlədiyimizi ifadə edən modeldir. int, double, bool, char, decimal kimi tiplər başlanğıc üçün value type kimi düşünülə bilər. Əsas qayda budur: value type başqa dəyişənə verildəndə dəyər kopyalanır.

```
int a = 10;
int b = a;

b = 99;

Console.WriteLine(a); // 10
Console.WriteLine(b); // 99
```

Bu kodda b dəyişəninə a-nın dəyəri verilir. Yəni b ayrıca 10 dəyəri alır. Sonra b-ni 99 edəndə a dəyişmir. Çünki a və b arasında paylaşılmış obyekt yoxdur.

Sətir	Nə baş verir?
int a = 10;	a adlı dəyişəndə 10 dəyəri saxlanılır.
int b = a;	a-nın dəyəri b-yə kopyalanır. b ayrıca 10 olur.
b = 99;	b dəyişir, amma a dəyişmir.

Bool ilə də eyni fikri görə bilərsiniz:

```
bool first = true;
bool second = first;

second = false;

Console.WriteLine(first); // true
Console.WriteLine(second); // false
```

Qısa qayda: Value type kopyalananda iki dəyişən bir-birindən ayrılır. Birini dəyişmək digərini dəyişmir.

4. Reference Type: obyektə istinadla işləmək

Reference type anlayışı data structure öyrənməyin əsas qapısıdır. Array, class, List, Dictionary kimi strukturlar reference davranışı ilə bağlıdır. Reference type dəyişəndə obyektin özü yox, obyektə gedən istinad saxlanılır kimi düşünə bilərsiniz.

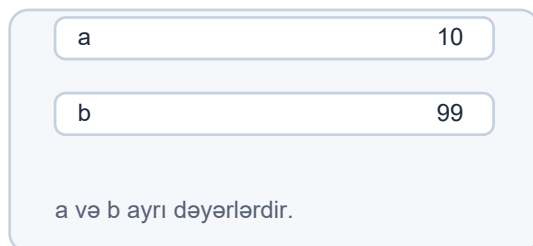
```
int[] arr1 = { 10, 20, 30 };
int[] arr2 = arr1;

arr2[1] = 999;

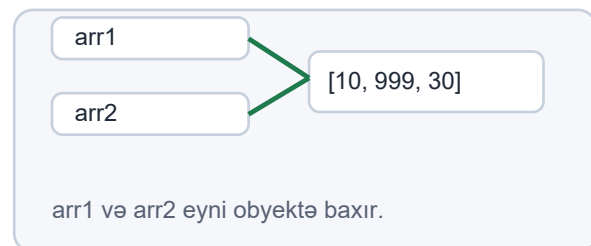
Console.WriteLine(arr1[1]); // 999
Console.WriteLine(arr2[1]); // 999
```

Burada `arr2 = arr1` yazanda array-in elementləri kopyalanmır. Heap-dəki eyni array obyektinə ikinci reference yaranır. Ona görə `arr2[1]` dəyişəndə array obyektinin içi dəyişir və `arr1` də həmin dəyişmiş obyektə göstərir.

Value copy



Reference copy



Ən vacib cümlə: Reference kopyalananda obyekt kopyalanmır. Eyni obyektə ikinci yol yaranır.

Bu davranış gələcəkdə class və object mövzusunda da qarşıya çıxacaq:

```
Student s1 = new Student();
Student s2 = s1;

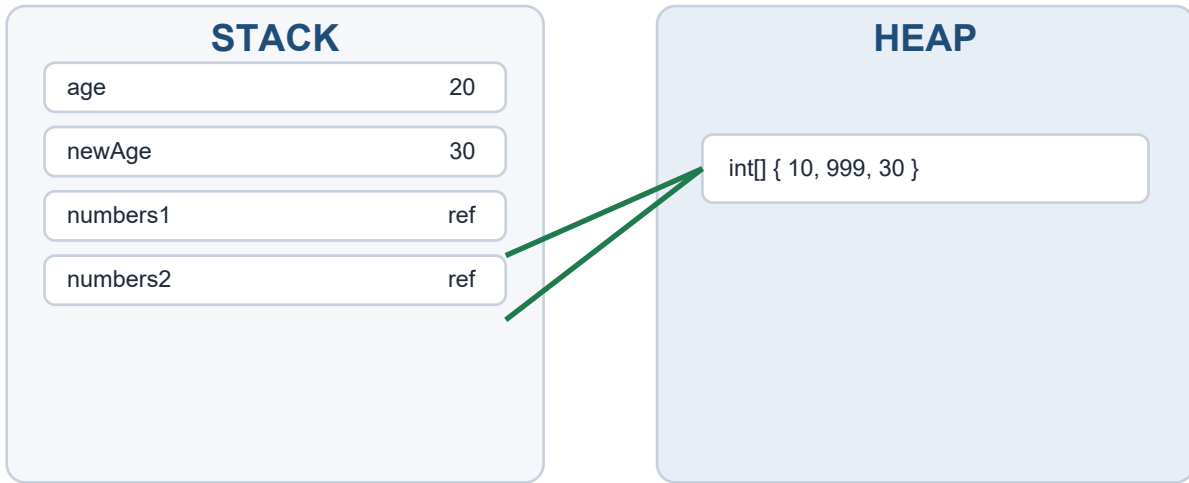
s2.Name = "Murad";

Console.WriteLine(s1.Name); // Murad
```

Əgər `s1` və `s2` eyni `Student` obyektini göstərsə, `s2` ilə dəyişilən property `s1` tərəfindən də görünür.

5. Stack və Heap: sadə mental model

Stack və heap mövzunu başlanğıcda çox ağır öyrənmək lazım deyil. Sənə indi lazım olan şey sadə modeldir: lokal value-lar və reference dəyişənlər metodun işləmə kontekstində olur, new ilə yaranan obyektlər isə heap tərəfində düşünülür.



Fikir: value-lar ayrıca kopyalanır, reference-lər isə heap-dəki eyni obyekti göstərə bilər.

Anlayış	Necə düşün?	Nümunə
Stack	Metod çağırışları və lokal dəyişənlər üçün nizamlı sahə kimi düşün.	<code>int age = 20;</code>
Heap	Obyektlərin yaşadığı daha geniş yaddaş sahəsi kimi düşün.	<code>new int[] { 1, 2, 3 }</code>
Reference	Obyektin özü deyil, obyektə gedən yol kimi düşün.	<code>arr</code> dəyişəni heap-dəki array-i göstərir.

Diqqət: Bu model öyrənmək üçündür. Real .NET runtime-da optimallaşdırmalar, obyekt header-i, generational GC kimi əlavə detallar var. Amma data structure-a başlamazdan əvvəl bu sadə model kifayətdir.

Mövzunu belə yadda saxla: value type davranışını başa düşmək üçün kopyalanan dəyərə bax. Reference type davranışını başa düşmək üçün dəyişənin hansı obyektə baxdığını düşün.

6. Assignment və Mutation fərqi

Tələbələrən ən çox qarışdırdığı yer budur: dəyişəni yeni obyektə yönləndirmək başqa şeydir, mövcud obyektin içini dəyişmək başqa şeydir.

```
int[] a = { 1, 2, 3 };
int[] b = a;

b[0] = 100;           // mutation
b = new int[] { 7 }; // assignment

Console.WriteLine(a[0]); // 100
Console.WriteLine(b[0]); // 7
```

Kod	Nə baş verir?
b[0] = 100;	b-nin göstərdiyi mövcud array obyektinin içi dəyişir. a da həmin obyekti göstərdiyi üçün a[0] 100 olur.
b = new int[] { 7 };	b artıq yeni array obyektinə baxır. a əvvəlki array-i göstərməyə davam edir.
Sadə bənzətmə: İki nəfərdə eyni Google Doc linki varsa, biri sənədin içini dəyişəndə o biri də görür. Amma biri tam başqa sənədə keçid alsaydı, əvvəlki sənəd o biri nəfərdə dəyişmir.	

Özünü yoxla

- b[0] = 100 obyektin içini dəyişir, yoxsa b dəyişənini yeni obyektə yönləndirir?
- b = new int[] { 7 } əvvəlki array-i silirmi?
- a əvvəlki array-i göstərməyə davam edərsə, əvvəlki array GC-yə namizəd ola bilərmi?

7. new keyword və obyekt yaranması

C#-da new yazanda runtime yeni obyekt yaratmaq üçün yaddaş ayırır və həmin obyektə reference qaytarır. Array yaratmaq, class obyektini yaratmaq və bir çox collection yaratmaq bu davranışla bağlıdır.

```
int[] numbers = new int[3];

numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
```

Bu kodda numbers dəyişəni array obyektinin özü deyil. numbers həmin array obyektinə baxan reference-dir. new int[3] isə 3 elementlik array obyektinin yaradılmasıdır.

Class nümunəsi:

```
User user = new User();
user.Name = "Ayse1";
```

Burada user dəyişəni User obyektinə reference saxlayır. new User() isə heap-də User obyektinin yaradılması kimi düşünülə bilər.

Gələcək mövzularla əlaqə: List, Dictionary, Stack, Queue kimi strukturları yaradanda da çox vaxt new istifadə edəcəksən. Bu strukturların daxilində hansı obyektlərin və array-lərin saxlandığını anlamaq üçün new anlayışı vacibdir.

8. Garbage Collector nə edir?

C#-da çox vaxt C/C++ kimi delete yazmırıq. Heap-də yaradılan obyektə artıq heç bir canlı reference qalmırsa, Garbage Collector həmin obyektin tutduğu yaddaşı sonradan təmizləyə bilər.



```
void CreateNumbers()
{
    int[] numbers = new int[1000];
    numbers[0] = 42;
}

// metod bitəndə numbers lokal reference-i artıq yoxdur.
// array-ə başqa reference yoxdursa, obyekt GC üçün namizəd olur.
```

Vacib məqam: GC obyektə reference qalmayan kimi dərhal işləməyə məcbur deyil. O, runtime-ın memory vəziyyətinə görə sonradan işləyə bilər. Ona görə 'GC dərhal silir' demək doğru deyil.

Sual	Düzgün cavab
GC nəyi təmizləyə bilər?	Artıq canlı reference ilə əlçatan olmayan heap obyektlərini.
GC dərhal işləyirmi?	Xeyr, runtime qərar verir.
C#-da memory leak ola bilər?	Bəli. Lazımsız obyektə reference saxlasan, GC onu təmizləyə bilməz.
Backend üçün nəticə: Cache, static list, uzunömürlü service və event subscription kimi yerlərdə lazımsız reference saxlamaq memory probleminə səbəb ola bilər.	

9. String haqqında qısa vacib qeyd

String reference type-dir, amma davranışı array və class obyektlərindən bir az fərqli görünə bilər. Çünki string immutable-dir. Immutable o deməkdir ki, string yaradıldıqdan sonra onun içi dəyişmir; dəyişiklik kimi görünən əməliyyat adətən yeni string yaradır.

```
string name = "Aysel";  
string upper = name.ToUpper();  
  
Console.WriteLine(name); // Aysel  
Console.WriteLine(upper); // AYSEL
```

name.ToUpper() name-in içini dəyişmir. Yeni string nəticəsi yaradır və upper dəyişəni həmin nəticəyə baxır. Buna görə string reference type olsa da, array kimi `arr[0] = ...` formasında içini dəyişə bilmərsən.

Niyə indi bilmək yaxşıdır: String mövzusu array-dən sonra ayrıca dərs kimi keçilməlidir. Amma bu qısa qeyd tələbənin 'reference type həmişə içdən dəyişir' kimi səhv nəticəyə gəlməsinin qarşısını alır.

10. Ən çox edilən səhvlər

Səhv fikir	Düzgün fikir
Reference type dəyişəni obyektin özüdür.	Dəyişən obyektə reference saxlayır; obyekt ayrıca düşünülür.
arr2 = arr1 array-i kopyalayır.	Bu sadəcə eyni array obyektinə ikinci reference yaradır.
b = new int[] əvvəlki obyektı dərhal silir.	Sadəcə b-ni yeni obyektə yönləndirir. Əvvəlki obyektə reference qalıb-qalmamasından asılıdır.
GC hər şeyi dərhal təmizləyir.	GC runtime qərarı ilə sonradan işləyir.
Stack/heap-i tam bilmədən data structure keçmək olar.	Sintaksis keçmək olar, amma dərin anlama üçün memory modeli lazımdır.

Kod oxuma məşqi 1

```
int x = 5;
int y = x;
y = 20;

Console.WriteLine(x);
Console.WriteLine(y);
```

Cavab: x = 5, y = 20.

Kod oxuma məşqi 2

```
int[] a = { 1, 2, 3 };
int[] b = a;
b[2] = 300;

Console.WriteLine(a[2]);
```

Cavab: 300. Çünki a və b eyni array obyektini göstərir.

11. Özünü yoxlama sualları

1. Dəyişən nədir? Ad, tip və dəyər anlayışlarını ayrıca izah et.
2. RAM nədir və diskdən nə ilə fərqlənir?
3. Value type kopyalananda nə baş verir?
4. Reference type kopyalananda nə baş verir?
5. Array niyə reference type nümunəsi kimi istifadə olunur?
6. Stack və heap-i sadə model kimi necə izah edərsən?
7. new keyword nə edir?
8. GC hansı obyektləri təmizləyə bilər?
9. Assignment və mutation fərqi nədir?
10. string reference type olsa da niyə array kimi dəyişmir?
11. C#-da memory leak necə yarana bilər?
12. Backend developer üçün bu mövzunu bilmək niyə vacibdir?

Praktiki tapşırıqlar

1. İki int dəyişəni ilə value copy nümunəsi yaz və nəticəni izah et.
2. İki bool dəyişəni ilə value copy nümunəsi yaz.
3. İki array dəyişəni ilə reference copy nümunəsi yaz.
4. Array-də bir reference ilə dəyişiklik edib digər reference-dən nəticəni göstər.
5. Aşağıdakı kodun nəticəsini əvvəl kağızda yaz, sonra C#-da yoxla.

```
int[] arr1 = { 5, 10, 15 };
int[] arr2 = arr1;

arr2[0] = 99;
arr1[2] = 77;

Console.WriteLine(arr1[0]);
Console.WriteLine(arr2[2]);
```

6. Bu kodun nəticəsini izah et:

```
int[] a = { 1, 2 };
int[] b = a;

b = new int[] { 9, 8 };

Console.WriteLine(a[0]);
Console.WriteLine(b[0]);
```

7. Student class yarat. s1 və s2 eyni obyekti göstərsin. s2.Name dəyişəndə s1.Name nəticəsini izah et.
8. Hər sətirdən sonra stack/heap vəziyyətini çəkəcəyin 5 sətirlik kod yaz.

12. Dərsin yekunu

Bir cümləlik yekun: Value type kopyalananda dəyər ayrılır, reference type kopyalananda obyekt paylaşılır; stack/heap modeli isə bu fərqi gözündə canlandırmağa kömək edir.

Bu mövzunu yaxşı başa düşsən, növbəti dərslər daha rahat olacaq. Array-in niyə fixed size olduğunu, List-nin niyə capacity saxladığını, LinkedList-in node-larla işlədiyini, Dictionary-nin niyə sürətli axtarış verdiyini anlamaq üçün yaddaş düşüncəsi lazımdır.

Növbəti mövzu üçün hazırlıq

- Array nədir?
- Index nədir?
- Array elementləri niyə ardıcıl saxlanılır kimi düşünülür?
- Array-də insert/delete zamanı niyə elementlər sürüşür?
- Array.Resize həqiqətən mövcud array-i böyüdülmü, yoxsa yeni array yaradır?